

LAB 01: MỘT SỐ THAO TÁC TRÊN ĐỒ THỊ

MỤC TIÊU

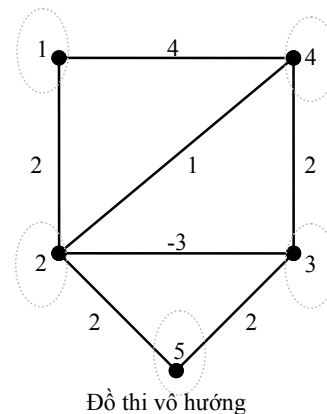
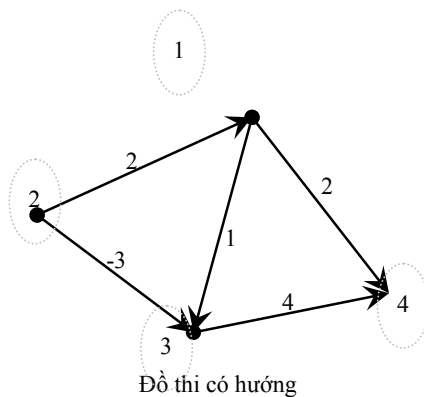
Hoàn tất bài thực hành này, sinh viên có thể:

- Biểu diễn đồ thị bằng ma trận kề.
- Kiểm tra tính hợp lệ của ma trận, Kiểm tra đồ thị vô hướng
- Thực hiện 1 số thao tác đơn giản trên đồ thị.

Thời gian thực hành: từ 120 phút đến 400 phút

TÓM TẮT

Ma trận kề là cấu trúc dữ liệu thường được sử dụng trong môn lý thuyết đồ thị để biểu diễn một đơn đồ thị. Một giá trị $a[i, j]$ trong ma trận (dòng i cột j của ma trận A) ứng trọng số của cung $a(i, j)$ trong đồ thị. Một giá trị đặc biệt được sử dụng cho phần tử $a[i, j]$ (thường là 0) để cho biết không có cung $a(i, j)$ trong đồ thị.



Theo định nghĩa trên thì đồ thị có hướng ở trên sẽ có ma trận kề là

	1	2	3	4
1	0	0	1	2
2	2	0	-3	0
3	0	0	0	4
4	0	0	0	0

Đối với đồ thị vô hướng, ta sẽ có giá trị $a[i, j] = a[j, i]$ (xem như có hai cung có hướng $u(i, j)$ và $u(j, i)$ trong đồ thị). Ma trận kề của đồ thị vô hướng ở trên là:

	1	2	3	4	5
1	0	2	0	4	0
2	2	0	-3	1	2
3	0	-3	0	2	2
4	4	1	2	0	0
5	0	2	2	0	0

Nhận xét: đường chéo chính trong đồ thị có giá trị 0 (do trong đồ thị không có khuyên)

Trong một số bài toán ta không quan tâm đến trọng số của đồ thị mà chỉ quan tâm là có tồn tại một cung $u(i, j)$ trong đồ thị hay không. Đối với trường hợp này, ta sử dụng giá trị 1 cho $a[i, j]$ để chỉ ra sự tồn tại của cung $a(i, j)$.

NỘI DUNG THỰC HÀNH

Cơ bản

Sinh viên đọc kỹ phát biểu bài tập và thực hiện theo hướng dẫn:

Cho một file text có cấu trúc như sau:

- Dòng đầu tiên chứa số nguyên n , cho biết số đỉnh của đồ thị
- n dòng tiếp theo, mỗi dòng chứa n số nguyên ứng với các phần tử trong ma trận kề.

Xây dựng lớp GRAPH cho đối tượng đồ thị.

Viết hàm khởi tạo từ 1 file text chứa ma trận kề của đồ thị. Kiểm tra tính hợp lệ của đồ thị vừa khởi tạo. Kiểm tra xem đồ thị có vô hướng hay không?

Phân tích

- Để biểu diễn ma trận kề, ta dùng một mảng (kiểu nguyên hay thực tùy vào yêu cầu đề bài). Như vậy lớp GRAPH có ít nhất 2 thuộc tính như sau

```
#define      MAX      100

class GRAPH
{
public:
    GRAPH(char* fname);
    ~GRAPH(void);
...
}
```

- Thao tác cần thực hiện: Khởi tạo ma trận từ file, Kiểm tra tính hợp lệ của ma trận, Kiểm tra đồ thị vô hướng.

Chương trình mẫu

dothi.txt

```
4
0    0    1    2
2    0   -3    0
0    0    0    4
0    0    0    0
```

GRAPH.h

```
#include "stdio.h"

#pragma once
#define      MAX      100

class GRAPH
{
public:
    GRAPH(char* fname);
    ~GRAPH(void);
private:
    int n; // số đỉnh của đồ thị
    int a[MAX][MAX]; // ma trận kề
public:
    void XuatDoThi(void);
    bool KiemTraDoThiHopLe(void);
    bool KiemTraDoThiVoHuong(void);
};
```

GRAPH.cpp

```

#include "GRAPH.h"

GRAPH::GRAPH(char* filename)
{
    FILE* f;
    f = fopen(filename, "rt");
    if (f == NULL){ //Cách xử lý nếu mở file bị lỗi ở đây là gì?
        n=0;
        return;
    }

    fscanf(f, "%d", &n);

    int i, j;
    for (i=0; i<n; i++){
        for (j=0; j<n; j++){
            fscanf(f, "%d", &a[i][j]); //Nếu trọng số là số thực thì sao?
        }
    }

    fclose(f);
}

GRAPH::~GRAPH(void)
{
}

void GRAPH::XuatDoThi(void)
{
    printf("%d\n", n);

    int i, j;
    for (i=0; i<n; i++){
        for (j=0; j<n; j++){
            printf( "%d\t", a[i][j]);

            printf( "\n", a[i][j]);
        }
    }
}

bool GRAPH::KiemTraDoThiHopLe(void)
{
    //Cho biết ý nghĩa thao tác sau
    if(n<=0){ //điều kiện này có chính xác chưa?
        return false;
    }

    //Điều kiện để ma trận hợp lệ là gì?
    int i;

    for (i=0; i<n; i++){
        if (a[i][i] != 0){
            return false;
        }
    }
    return true;
}

bool GRAPH::KiemTraDoThiVoHuong(void)
{
    int i, j;
    for (i=0; i<n; i++){
        for (j=0; j<n; j++){
            // Cho biết ý nghĩa 2 vòng for này.

```

```

        if (a[i][j] != a[j][i]){
            return false;
        }
    }

    return true;
}

```

LTD.T.cpp

```

#include "GRAPH.h"

void main()
{
    GRAPH g("dothi.txt");

    if(g.KiemTraDoThiHopLe()){
        g.XuatDoThi();
    }
    else{
        printf("Do thi không hop le");
    }

    if(g.KiemTraDoThiVoHuong()){
        printf("Do thi vo huong.\n");
    }
    else{
        printf("Do thi co huong.\n");
    }
}

```

Yêu cầu

1. Biên dịch đoạn chương trình nêu trên.
2. Cho khởi tạo đồ thị từ tập tin dothi1.txt có nội dung mô tả đồ thị thứ 2 ở phần tóm tắt lý thuyết. Cho biết kết quả xuất ra màn hình.
3. Giải thích ý nghĩa các hàm trong chương trình mẫu và chi tiết từng dòng lệnh.

Áp dụng – Nâng cao

1. Thêm phương thức để đếm số lượng cạnh của đồ thị.

H.D: Phân biệt 2 trường hợp đồ thị có hướng và vô hướng.

2. Thêm phương thức để in ra bậc của từng đỉnh.

H.D: Bậc của 1 đỉnh trong đồ thị vô hướng là tổng số các cạnh kề với đỉnh đó. Khuyến được tính 2 lần.

Bậc nửa ngoài/trong của 1 đỉnh trong đồ thị hữu hướng là số cạnh đi ra/vào đỉnh đó. Tổng nửa bậc ngoài và trong của đỉnh là bậc của đỉnh.

3. Thêm phương thức để đếm số lượng đỉnh bậc chẵn.
4. Thêm phương thức để đếm số lượng đỉnh bậc lẻ.
5. Thêm phương thức để đếm số lượng đỉnh treo.

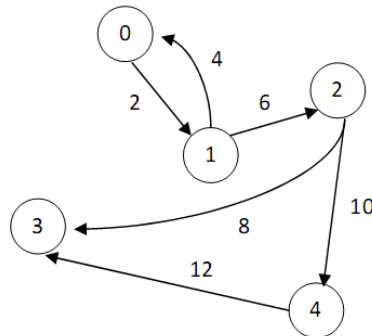
H.D: Đỉnh treo là đỉnh có bậc bằng 1.

6. Thêm phương thức để đếm số lượng đỉnh cô lập.

H.D: Đỉnh cô lập là đỉnh có bậc bằng 0.

BÀI TẬP THÊM

1. Cải tiến hàm `bool KiemTraDoThiVoHuong(void)`
2. Viết lại chương trình cho phép kiểm tra tính hợp lệ của ma trận trước khi khởi tạo. Nêu phương án xử lý cho trường hợp file nhập liệu không hợp lệ.
3. Biểu diễn đồ thị bằng ma trận liên thuộc. Viết hàm chuyển đổi giữa 2 loại ma trận (các trường hợp thông thường). Viết các thao tác xử lý tương ứng. Nhận xét ưu và khuyết điểm của 2 cách biểu diễn.
4. Một cách biểu diễn đồ thị khác là biểu diễn bằng danh sách đỉnh kề, thường được dùng cho trường hợp đồ thị ít cạnh. Mỗi đỉnh sẽ có 1 danh sách các đỉnh kề của nó. Ví dụ:



Đỉnh	Danh sách đỉnh kề
0	→ 1 2 → ●
1	→ 0 4 → 2 6 → ●
2	→ 3 8 → 4 10 → ●
3	→ ●
4	→ 3 12 → ●

Cài đặt cách biểu diễn trên. Viết hàm khởi tạo từ ma trận kề. Viết các thao tác xử lý tương ứng. Nhận xét ưu và khuyết điểm của các này so với 2 cách biểu diễn bằng ma trận.